FC Fluxo de Caixa

DOCUMENTO 03

Arquitetura alvo, C4 e fluxos de dados

Modelos C4 de contexto a código, fluxos, padrões de integração e implantação na AWS

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

Arquitetura 3

Arquitetura-alvo 4

C4 — Nível 1: contexto 9

C4 — Nível 2: containers 10

C4 — Nível 3: componentes do lancamentos-service 12

C4 — Nível 3: componentes do consolidado-service 14

C4 — Nível 4: registrar lançamento 16

Fluxos de dados 18

Padrões de integração 24

Deployment na AWS 31

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/README.MD

Arquitetura

Visão de solução do Fluxo de Caixa. Os nomes de serviços, filas, portas e variáveis são os do blueprint.

Documento	Conteúdo
C4 nível 1 — contexto	Pessoas e sistemas externos
C4 nível 2 — containers	Serviços, bancos, mensageria, cache, identidade
C4 nível 3 — lançamentos	Componentes hexagonais do `lancamentos-service`
C4 nível 3 — consolidado	Componentes hexagonais do `consolidado-service`
C4 nível 4 — registrar lançamento	Código-alvo do caso de uso de escrita
Arquitetura-alvo	Estilos, padrões e trade-off de cada um
Fluxos de dados	Sequências: escrita, outbox, consolidação, cache, estorno, falha, DLQ
Padrões de integração	EIP entre a borda e os serviços, e entre lançamentos e consolidado
Deployment AWS	CloudFront, API Gateway, ALB interno, VPC, três ambientes e DR em `us-east-1`
Structurizr DSL	Workspace com context, containers, components, dynamic e deployment

Os diagramas Mermaid usam a sintaxe C4 (`C4Context` , `C4Container` , `C4Component`), que o GitHub renderiza. O workspace Structurizr é a fonte editável. A imagem `structurizr/cli:latest` de novembro de 2026 só imprime o aviso de migração; a tag `2025.03.28` valida e exporta este arquivo:

```
docker run --rm \
  -v "$PWD/docs/03-arquitetura/structurizr":/usr/local/structurizr \
  structurizr/cli:2025.03.28 \
  validate -w /usr/local/structurizr/workspace.dsl

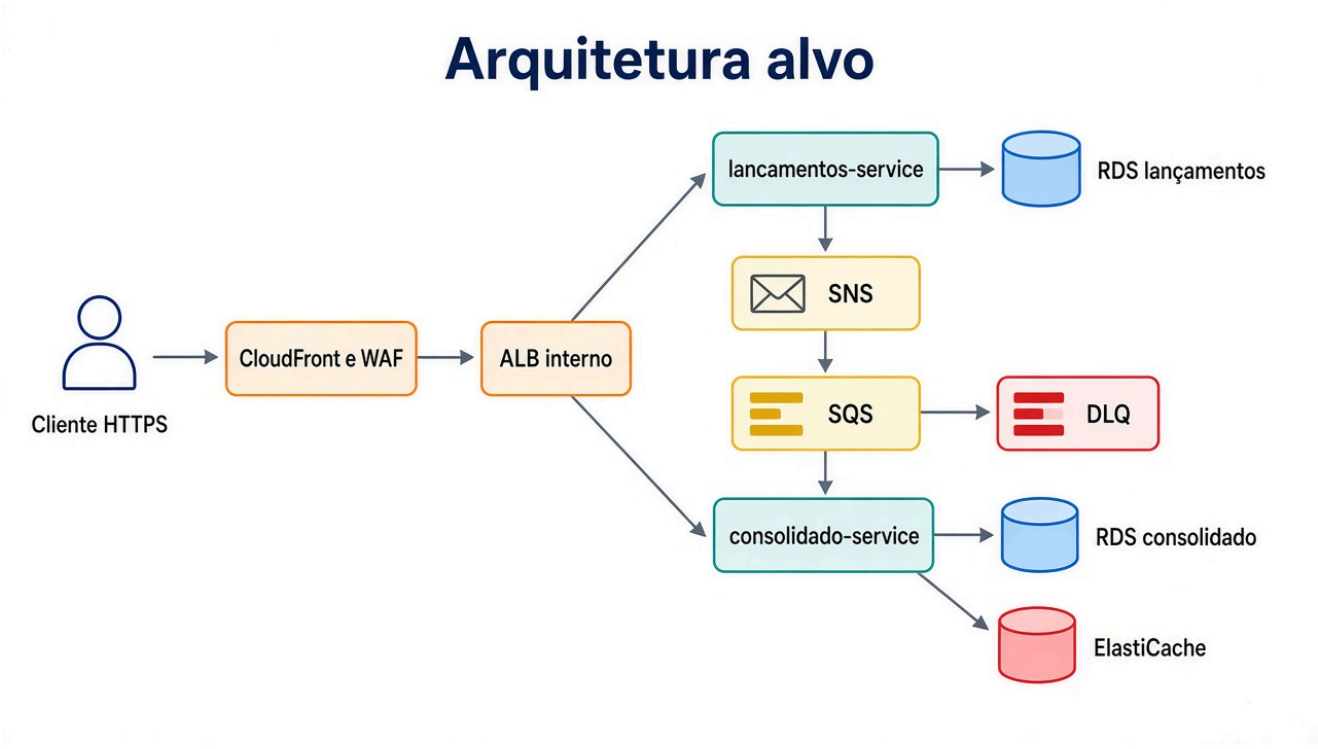
docker run --rm \
  -v "$PWD/docs/03-arquitetura/structurizr":/usr/local/structurizr \
  structurizr/cli:2025.03.28 \
  export -w /usr/local/structurizr/workspace.dsl \
  -format plantuml -o /usr/local/structurizr
```

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/ARQUITETURA-ALVO.MD

Arquitetura-alvo

1. Visão

Dois microsserviços alinhados a bounded contexts, com CQRS entre a escrita de lançamentos e a leitura do saldo. A integração é assíncrona. O POST não espera o relatório. A durabilidade do fato fica no Postgres de lançamentos; a disponibilidade do relatório em pico fica numa projeção pequena, cache e duas tasks.



O RNF crítico cabe neste desenho: a seta entre os serviços aponta só para a direita e termina numa fila, não numa chamada bloqueante. A conta de 50 req/s, 2 tasks e 0,16 conexões ativas está em contexto e requisitos.

2. Estilos e padrões

Cada padrão abaixo está adotado. O trade-off é o preço que aceitamos pagar.

Microsserviços por bounded context

Dois deploys, dois bancos, dois ciclos de vida. Lançamentos pode ser implantado sem tocar no consolidado.

Trade-off. Operação duplicada (pipeline, alarme, RDS). Um monolito modular daria a mesma fronteira de código com um deploy só, e atenderia o isolamento lógico se o módulo de escrita não chamasse o de leitura. Não atenderia o isolamento de *processo e de banco* que o enunciado pede quando "o sistema de consolidado cair": no mesmo processo, um vazamento de pool ou um crash derruba os dois. Serverless foi descartado pelo perfil de conexões longas com Postgres e pelo custo fixo de VPC em Lambda. Detalhe no ADR 0002.

CQRS

A escrita persiste o fato. A leitura serve uma projeção com outro esquema (`saldos_diarios` não é a tabela `lancamentos`).

Trade-off. Dois modelos para manter e uma janela de consistência eventual (p95 < 5 s). Em troca, o GET de pico não faz join no extrato, e o esquema da leitura pode ganhar índice e cache sem migrar a trilha de auditoria. ADR 0003.

Event-driven

O fato vira `LancamentoRegistrado` v1. Novos consumidores (notificação, auditoria externa, categoria) assinam o tópico com fila própria, sem o de escrita conhecê-los.

Trade-off. Depuração deixa de ser uma stack única. Exige correlation id, outbox e consumidor idempotente. Sem isso, evento é perda silenciosa.

Transactional outbox

Lançamento e linha em `outbox_eventos` no mesmo commit. Relay com `FOR UPDATE SKIP LOCKED`, lote 100, intervalo 500 ms.

Trade-off. Latência extra de até um ciclo do relay no caminho do evento (não no caminho do POST) e uma tabela para operar. Dual-write (commit e publish separados) perde evento ou publica evento de transação revertida. CDC com Debezium entrega o log, com um cluster a mais que este volume não paga. ADR 0004.

Consumidor idempotente

`eventos_processados(event_id PK)` na mesma transação da soma. A fila é at-least-once e Standard (sem ordem). A soma é comutativa, então a ausência de FIFO é aceitável.

Trade-off. Toda mensagem custa um insert de id. É a alternativa barata a “processar exatamente uma vez”, que o SQS não oferece.

Hexagonal

Domínio em Java puro, casos de uso nas portas, AWS e JPA nos adaptadores. ArchUnit no build.

Trade-off. Mais pacotes e mapeamentos. O ganho é poder testar estorno e dinheiro sem subir Spring, e trocar SNS por LocalStack sem editar regra. ADR 0008.

Cache-aside

O consolidado consulta Redis no local e Valkey no ElastiCache e, no miss, o Postgres, depois preenche o cache. A chave carrega a versão corrente do comerciante (`consolidado:versao:{id}`), incrementada a cada evento. O TTL de 30 s (`app.cache.ttl`) só apaga chave órfã; a leitura seguinte já usa a versão nova.

Trade-off. Memória e um modo de falha a mais. Se o Redis cair, a leitura segue no Postgres: em 50 req/s com miss total são ~2 conexões ativas, dentro do `db.t4g.small`. O cache existe para o p95 < 100 ms, não para a sobrevivência. ADR 0012.

Database per service

Cada serviço é dono do seu esquema e, em produção, da sua instância RDS. Não há `JOIN` entre lançamento e saldo, nem usuário de banco compartilhado.

Trade-off. Duas instâncias Multi-AZ custam mais do que um RDS com dois databases (a conta está no documento 08). O ganho é o de falha: esgotar conexão, disco ou IOPS do consolidado não tira o POST do ar. Em dev AWS as instâncias podem ser menores e Single-AZ; no Docker, um Postgres com dois databases basta.

Retry com backoff e jitter, timeout, DLQ

- Relay: retenta o publish, incrementa `tentativas`, não segura a thread do POST.
- SQS: visibility 60 s, até 5 recebimentos, depois `consolidado-lancamentos-dlq`.
- Redis e JWKS: timeout curto e retry com jitter. Falha de Redis degrada para o banco.
- Não se retenta 4xx de domínio. Estorno inválido não melhora na segunda tentativa.

Trade-off. Retry sem idempotência duplicaria saldo. Os dois padrões andam juntos. ADR 0014.

Circuit breaker

Há dois mecanismos com o mesmo nome, e eles não se misturam.

- **De deploy (ECS).** Circuit breaker com rollback automático se as tasks novas não ficam saudáveis. ADR 0020.
- **De aplicação (Resilience4j 2.4.0).** CircuitBreaker, Retry com jitter e TimeLimiter no cliente SNS do relay. Circuito aberto adia o lote; o evento fica na outbox. No consolidado, CircuitBreaker no Redis com fallback no Postgres, e Bulkhead no consumidor. Não existe circuito entre lançamentos e consolidado porque não existe chamada entre eles. ADR 0023.

Trade-off. O POST continua sem chamar o SNS. O circuito protege o relay, não a latência do 201. Timeout mal calibrado no Redis transforma lentidão em fallback, que é o comportamento desejado. Boot 4.1 fica bloqueado até o starter do Resilience4j declarar suporte.

Bulkhead

Pools JDBC separados, serviços ECS separados, filas separadas, bancos separados. Um consumidor lento não ocupa conexão do POST. O teto do pool é 10 no lançamentos e 20 no consolidado, por task.

Trade-off. Conexão ociosa (`minimumIdle` 2) custa slot no `max_connections`. O número foi escolhido na análise de capacidade para sobrar folga, não para “usar o banco inteiro”.

DLQ

Quinta falha de processamento vai para `consolidado-lancamentos-dlq`. A fila principal segue. Reprocessar é uma ação operada (corrigir o consumidor e reenviar), descrita no fluxo de DLQ e no runbook.

Trade-off. Mensagem na DLQ é saldo atrasado para aquele evento até alguém agir. O alarme é imediato. Sem DLQ, um poison message seguraria a partição lógica atrás de retries infinitos — no Standard não bloqueia as outras, mas queimaria receive e esconderia o erro.

API versionada e RFC 9457

Base `/api/v1`. Erro em `application/problem+json`. Evolução do evento por `eventVersion`. ADR 0017.

Trade-off. URIs de problema e versões são contrato para sempre. O ganho é cliente que distingue 409 de estorno de 400 de payload sem parsear texto livre.

Borda CloudFront, WAF e ALB interno, JWT no serviço

CloudFront é a única entrada pública. O WAF é de escopo `CLOUDFRONT`. O ALB é interno, alcançado por VPC Origin, e roteia `/api/v1/lancamentos*` e `/api/v1/consolidados*` para target groups diferentes. Health check em `/actuator/health/readiness`. `/api/*` não é cacheado. O JWT é validado no serviço, igual no Keycloak local e no Cognito.

Trade-off. O CloudFront não reduz carga de API, porque a resposta autenticada não se cacheia. O ganho é TLS no edge, Shield Standard e ALB sem IP público. API Gateway continua um hop e uma conta a mais, e um authorizer só na borda deixaria o serviço nu se alguém alcançasse a task. ADR 0022.

BFF / API Gateway — considerado e não adotado

Um BFF faria sentido com um frontend agregando saldo e extrato numa tela, escondendo os dois serviços. Não há frontend nesta entrega. Agregar no BFF agora criaria um terceiro serviço sem dono de negócio. O cliente chama os dois hosts paths no mesmo ALB. Quando existir canal web, o BFF entra na frente sem mudar o domínio — está no roadmap.

Strangler Fig

Usado na transição a partir de planilha ou ERP, não no dia a dia dos dois serviços novos. Os dois serviços já nascem no contorno certo; não há monolito nosso para estrangular. O padrão está especificado no documento 09 para o legado do comerciante.

Idempotência na borda HTTP

`Idempotency-Key` opcional, escopada pelo comerciante. Mesma chave e mesmo corpo: 201 na primeira vez, 200 na repetição. Corpo diferente: 409. ADR 0014.

Lançamentos imutáveis

Correção é estorno. A trilha é a própria tabela, append-only. ADR 0015.

Dinheiro

`BigDecimal` e `NUMERIC(19,2)` dentro; string de duas casas no evento. ADR 0016.

3. O que a arquitetura-alvo deliberadamente não é

Tentação	Por que não
Event sourcing do lançamento	O estado cabe numa linha. O evento de integração não precisa reconstruir o agregado. Reprocessar saldo a partir do tópico é evolução, não requisito.
Saga	Não há transação distribuída para compensar. O estorno é um fato novo, não um rollback do POST.

Tentação	Por que não
Service mesh	Dois serviços atrás de um ALB não pagam o custo de Istio/App Mesh. mTLS pode ser feito na evolução com VPC Lattice ou mesh quando houver dezenas de serviços.
Kafka / MSK	O volume da premissa (20 evt/s, 2 milhões/mês) é ruído para um cluster. SNS+SQS entrega DLQ, retenção e fan-out. ADR 0005.

4. Como as qualidades caem nos padrões

Qualidade	Padrão que a carrega
Lançamentos sobrevive ao consolidado	Assíncrono + outbox + fila + banco separado
50 req/s com perda baixa	Projeção + cache + 2 tasks com ρ ≈ 17% + WAF acima do pico
Auditoria	Append-only e estorno
Isolamento de tenant	Claim no token, filtro em toda query
Evolução do contrato	`/api/v1` e `eventVersion`
Custo proporcional	Fargate, RDS small, fila gerenciada; NAT e endpoint são o custo fixo a vigiar

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL1-CONTEXTO.MD

C4 — Nível 1: contexto

O comerciante usa o Fluxo de Caixa para registrar movimentos e ler o saldo. O sistema delega a autenticação a um provedor de identidade e, na implantação AWS, se apoia em serviços gerenciados de mensageria, dados e observabilidade. Esses serviços aparecem no nível 2 e na implantação; aqui ficam de fora para o diagrama mostrar só quem fala com quem no negócio.



Ator ou sistema	Responsabilidade na fronteira
Comerciante	Pessoa ou sistema dele (PDV, app) que chama a API. Não escolhe o tenant: o token já o traz.
Fluxo de Caixa	Dois serviços. Escrita durável de lançamentos e leitura eventual do saldo.
Provedor de identidade	Autentica, emite escopos `lancamentos:write`, `lancamentos:read`, `consolidado:read` e o claim `comerciante_id`.
Caixa legado	Fora do caminho de produção do dia zero. Entra na arquitetura de transição.

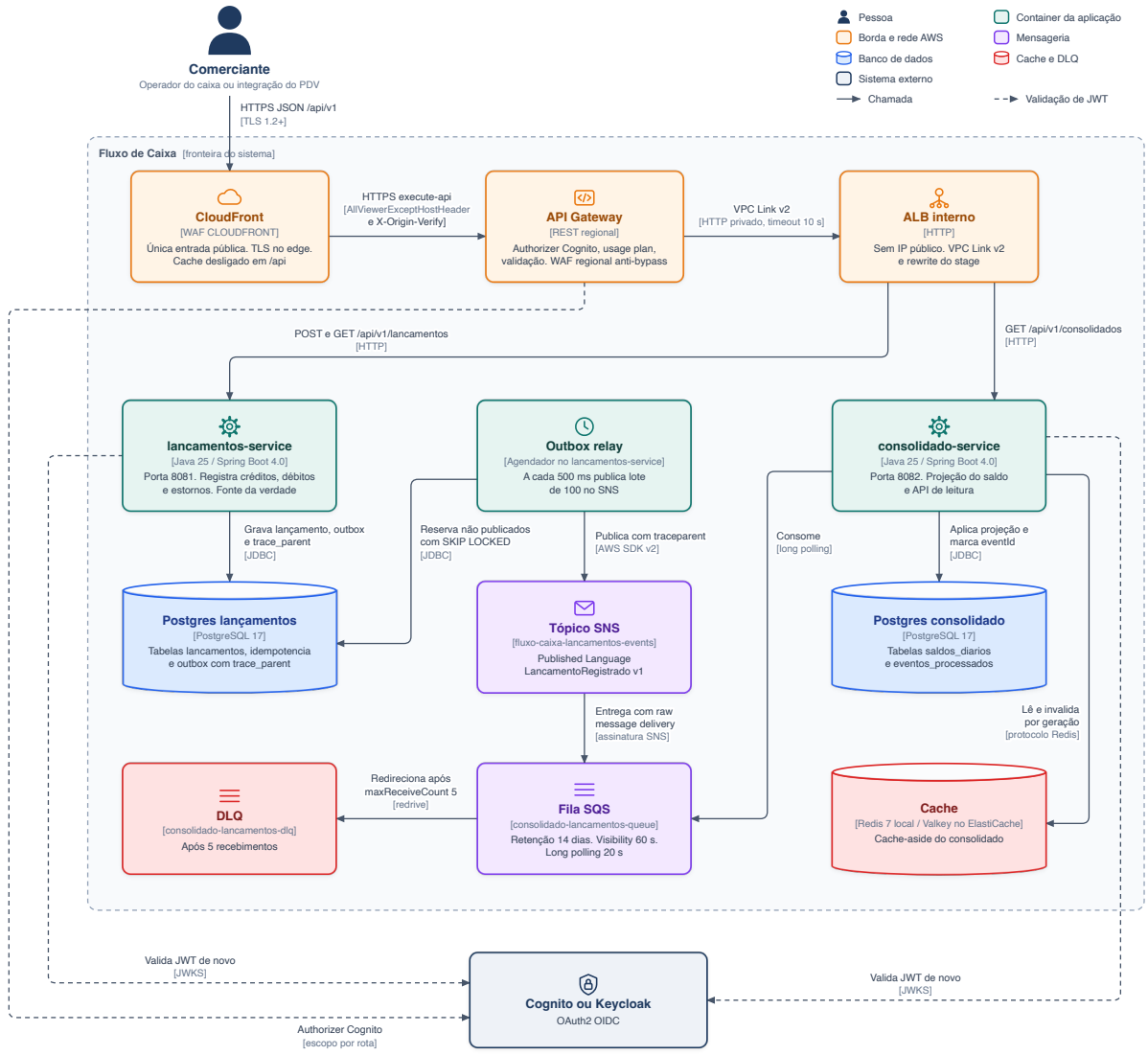
O diagrama de contexto de propósito não mostra SNS, RDS nem fila. Se eles aparecem aqui, o leitor confunde mecanismo com capacidade.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL2-CONTAINERS.MD

C4 — Nível 2: containers

Dois processos, dois bancos, um tópico, uma fila e uma DLQ. Cache só no consolidado. Identidade fora dos dois serviços.

C4 — Nível 2: containers do Fluxo de Caixa



O relay está desenhado como container lógico. No deploy ele é um agendador dentro do processo do `lancamentos-service`, não um terceiro serviço. Separá-lo no diagrama deixa visível que a publicação não está no caminho do POST. Subir um processo exclusivo de relay só vale se o agendamento competir com a API por CPU — não é o caso em 20 escritas/s.

Container	Escala	Estado
CloudFront + WAF	Distribuição global, PoP GRU	Sem estado de negócio. `/api/*` não é cacheado.

Container	Escala	Estado
API Gateway REST	Regional, stage `v1`	Sem estado de negócio. Rejeita JWT inválido antes da VPC.
ALB interno	Um, nas subnets privadas	Sem IP público. Dois target groups. Aceita o VPC Link.
`lancamentos-service`	2+ tasks ECS em prod	Stateless. Estado no RDS de lançamentos.
`consolidado-service`	2+ tasks ECS em prod	Stateless. Estado no RDS de consolidado e no cache.
Postgres (cada um)	RDS Multi-AZ em produção	Fonte durável.
SNS, SQS, DLQ	Gerenciados	Buffer entre os contextos.
Redis / Valkey	Réplica com failover em produção	Descartável. Perda do cache só aumenta latência.
Cognito / Keycloak	Gerenciado / container local	Fora do domínio.

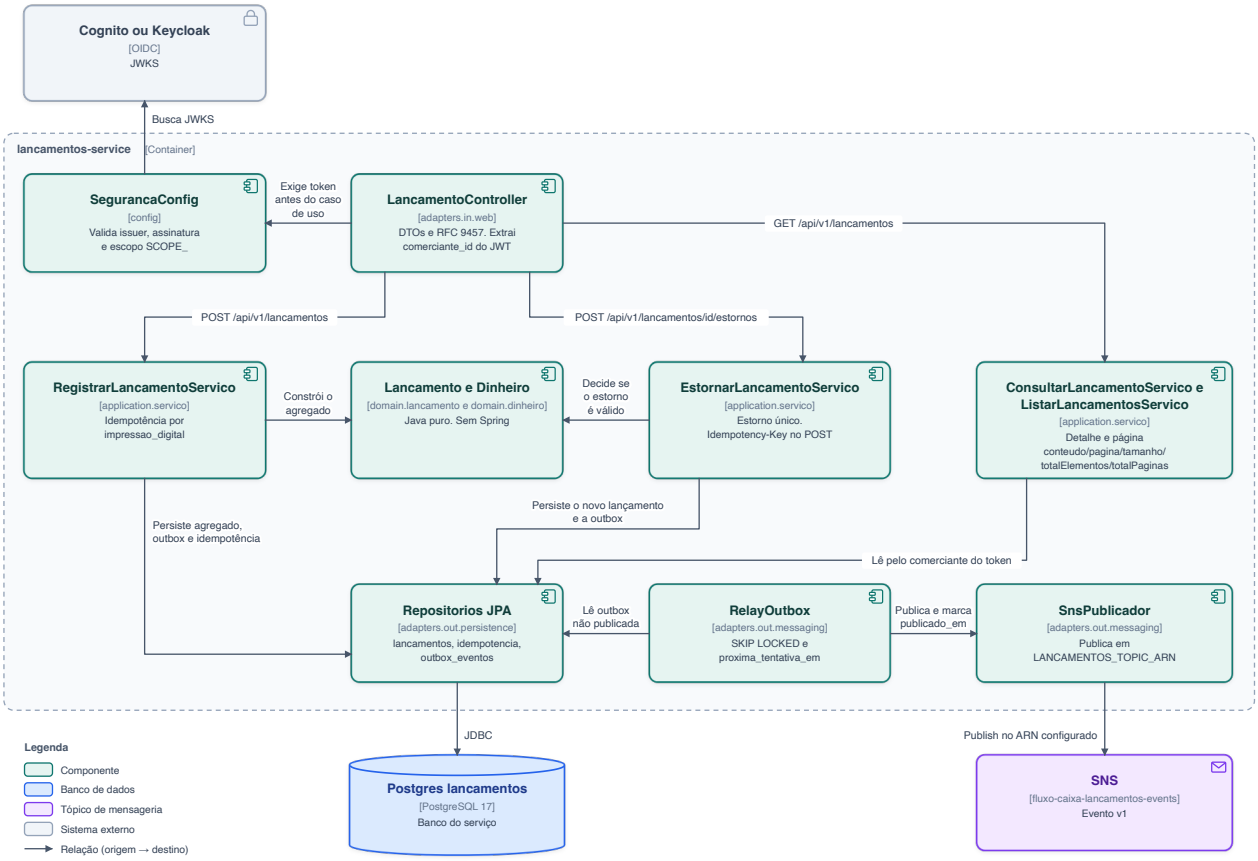
Portas locais: lançamentos 8081, consolidado 8082, Postgres 5432, Redis 6379, LocalStack 4566, Keycloak 8080, Grafana 3000, OTLP 4317 e 4318. Na AWS o cliente fala só com o CloudFront em 443. O API Gateway é regional e não é a URL que se publica. O ALB é interno, sem IP público, e as portas dos serviços ficam na rede privada. O relay continua sendo um agendador dentro do `lancamentos-service`, desenhado à parte para deixar visível que a publicação não está no caminho do POST. Com `enable_api_gateway = false`, o CloudFront volta a falar com o ALB pela VPC Origin e o container do gateway sai do caminho.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL3-LANCAMENTOS.MD

C4 — Nível 3: componentes do lancamentos-service

Arquitetura hexagonal no pacote `br.com.fluxocaixa.lancamentos`. Regra verificada por ArchUnit: `domain` não depende de Spring nem de `adapters`; `application` não depende de `adapters`.

C4 — Nível 3: componentes do lancamentos-service



Porta	Implementação	Observação
Entrada HTTP	`adapters/in/web`	Única porta síncrona. Não há listener de mensagem neste serviço.
Entrada de aplicação	`application.port.in` implementado em `application.servico`	O controller não calcula estorno.
Saída de persistência	JPA em `adapters.out.persistence`	Uma transação cobre lançamento, idempotência e outbox.
Saída de mensagem	`RelayOutbox` e `SnsPublicador`	Fora da transação do POST. Falha aqui não desfaz o 201. O nome do tópico não está no código.

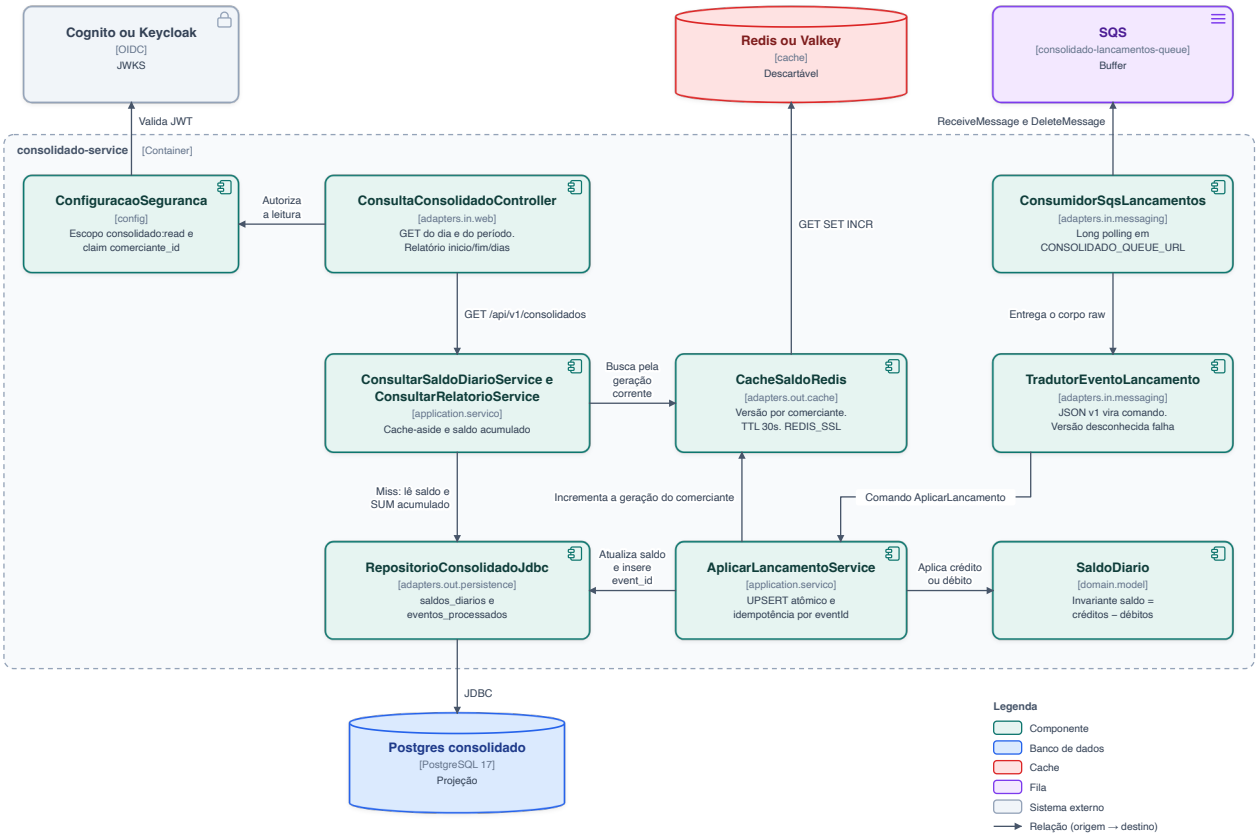
O domínio não importa `software.amazon.awssdk` nem `org.springframework`. O teste de arquitetura quebra o build se isso acontecer.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL3-CONSOLIDADO.MD

C4 — Nível 3: componentes do consolidado-service

Pacote `br.com.fluxocaixa.consolidado`. O HTTP só lê. Quem escreve a projeção é o listener SQS.

C4 — Nível 3: componentes do consolidado-service



Decisão local	Efeito
ACL no `TradutorEventoLancamento`, não no agregado	`SaldoDiario` não conhece JSON.
`eventos_processados` na mesma transação do UPSERT	`ON CONFLICT DO NOTHING` no evento e `ON CONFLICT DO UPDATE` somando em `saldos_diarios`. Reentrega não duplica saldo. Não há lock otimista.
Falha de parse ou versão ignorada	A mensagem volta após o visibility timeout e, na quinta vez, cai na DLQ. Não se aplica “no melhor esforço”.
Redis fora	O caso de uso lê o Postgres. Latência piora, disponibilidade da leitura permanece, dentro do que o RDS aguenta (ver capacidade).
Geração de cache	Um estorno em data antiga invalida acumulado e período daquele comerciante sem varrer chave por chave.

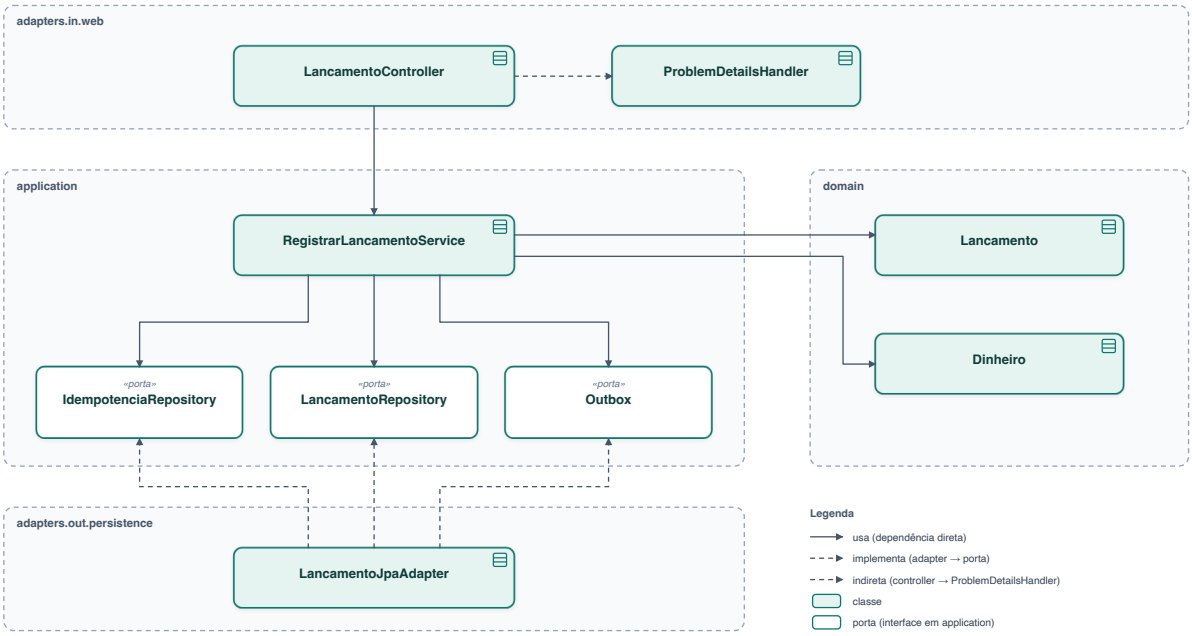
Não há porta de saída para o `lancamentos-service`. O consolidado não pergunta “qual é o lançamento?”. Se o evento não basta, o contrato está incompleto e a correção é versão nova do evento, não uma chamada síncrona de volta.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL4-REGISTRAR-LANCAMENTO.MD

C4 — Nível 4: registrar lançamento

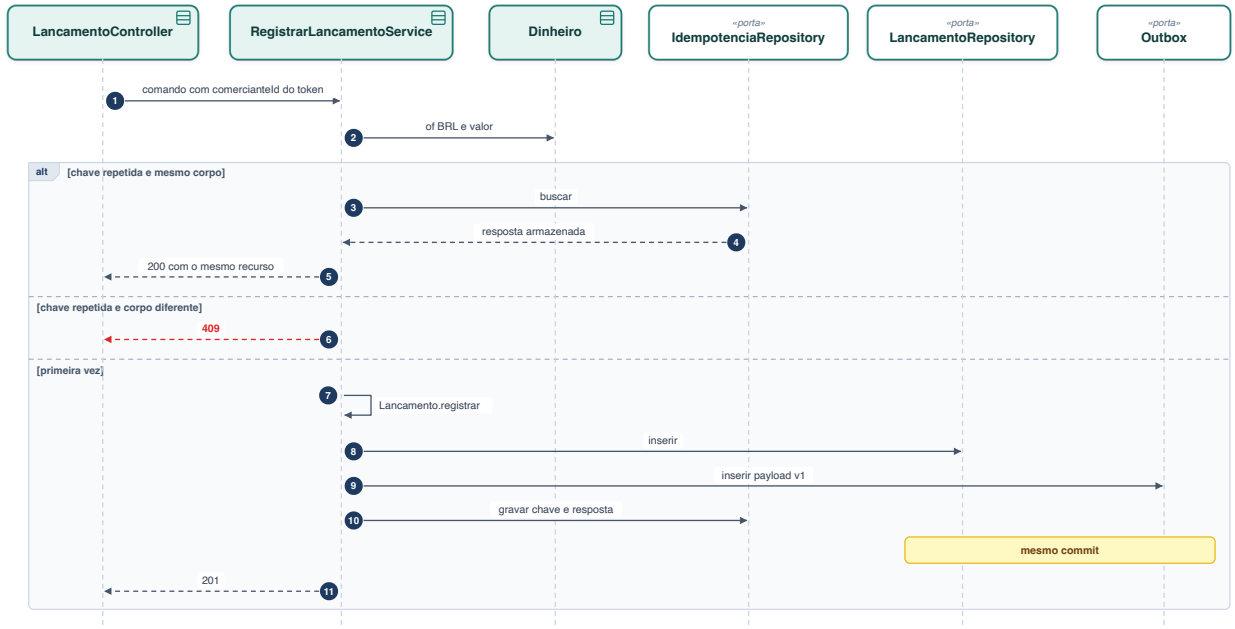
Nível de código do caso de uso que segura o RNF de independência. As classes abaixo são o alvo da implementação em `services/lancamentos-service`; os nomes de pacote são os do blueprint. Não há framework no miolo.

C4 — Nível 4: classes do registrar lançamento



Sequência interna, já com a transação no lugar certo:

Sequência interna do registrar lançamento



Pontos que o código não pode “simplificar”:

- O controller lê `comerciante_id` do `Jwt` e não de um campo do JSON. Se o JSON trouxer o campo, ele é ignorado.
- `Dinheiro.of` rejeita escala e sinal antes de qualquer `INSERT`.
- `Outbox.inserir` participa da transação do repositório. Publicar SNS dentro desse método quebraria o desenho: o SDK é chamado só pelo relay, depois do commit.
- A resposta 201 devolve `valor` como string de duas casas, alinhada ao evento e ao ADR 0016. O request continua aceitando número JSON, como no exemplo do blueprint.

O estorno segue o mesmo desenho, com `Lancamento.estornar(original)` no domínio devolvendo ou uma falha de invariante ou um novo `Lancamento`. A checagem “já estornado” é repetida pelo índice único: duas requisições paralelas não criam dois estornos.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/FLUXOS-DE-DADOS.MD

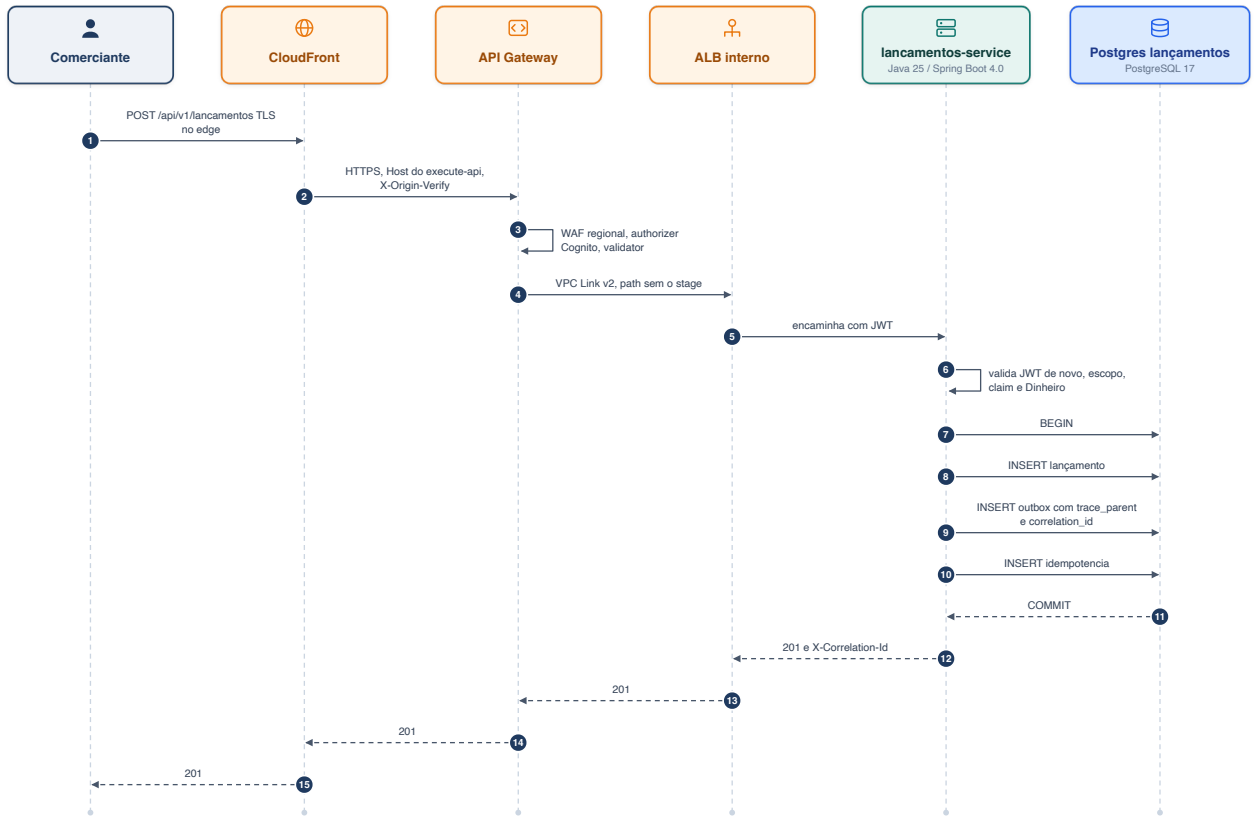
Fluxos de dados

Nomes e tempos seguem o blueprint: tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ `consolidado-lancamentos-dlq`, relay a cada 500 ms, visibility 60 s, `maxReceiveCount` 5, atraso de consolidação p95 < 5 s.

1. Registrar lançamento

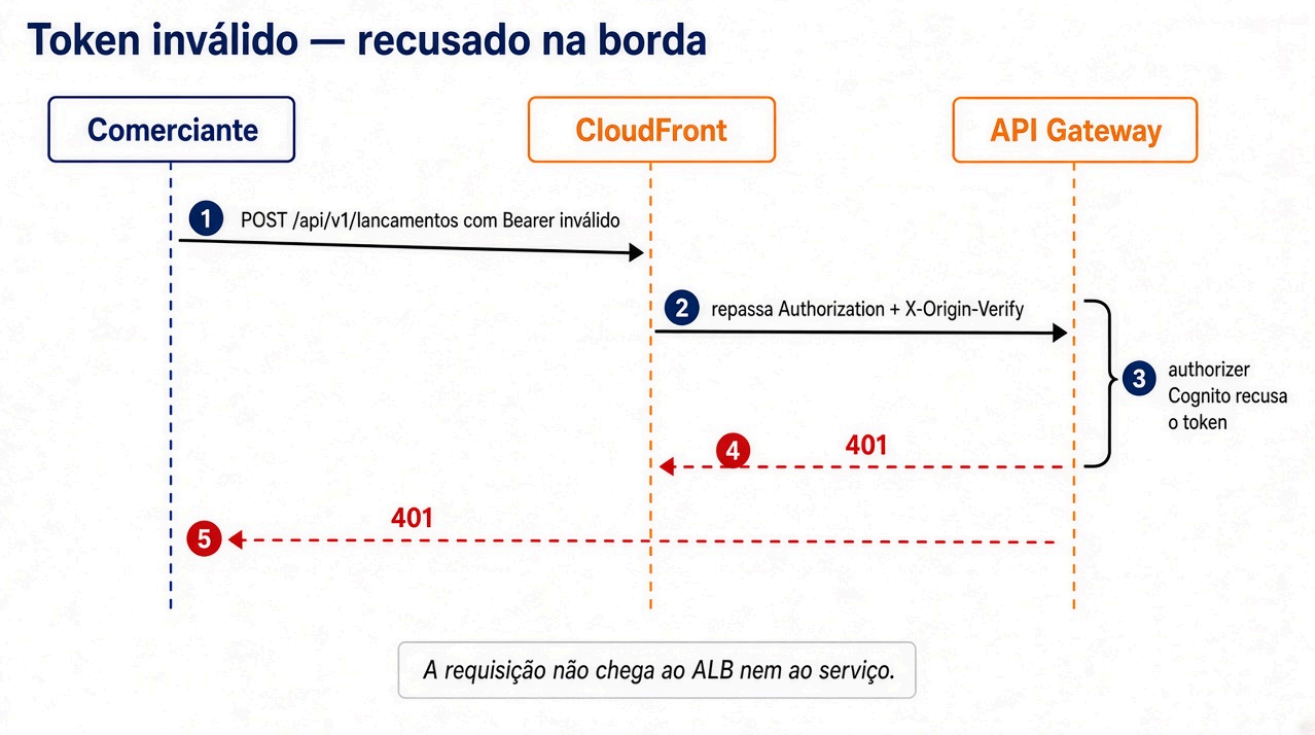
O POST termina no commit local. A publicação é outro fluxo.

Registrar lançamento



Se o header `Idempotency-Key` repetir a mesma requisição, o fluxo para no `SELECT` da idempotência e devolve 200, sem segundo insert.

Token inválido não chega na task:



O WAF regional responde 403 se o header `X-Origin-Verify` não for o atual nem o anterior. Esse caso é chamada direta ao `execute-api`, não o comerciante passando pelo CloudFront. O serviço continua validando o JWT de quem passou: o gateway não substitui o resource server.

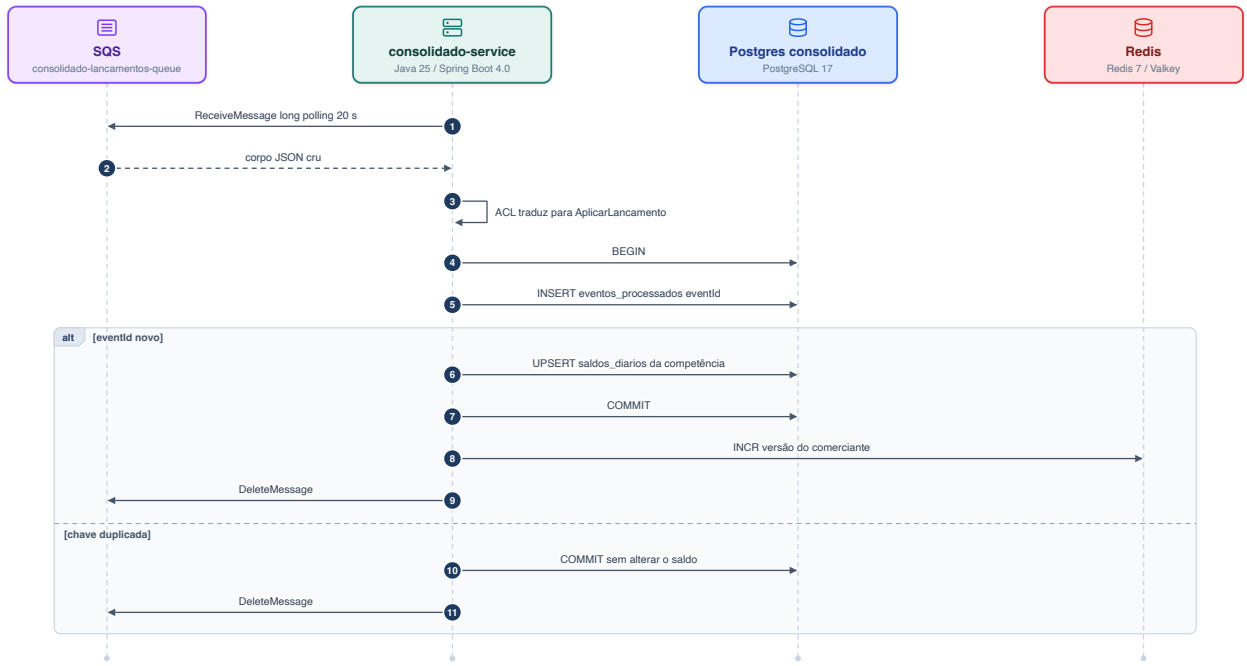
2. Outbox relay



Falha no `Publish`: a transação do lote não marca `publicado_em`, `tentativas` sobe, o próximo ciclo tenta de novo. Duas tasks não pegam a mesma linha por causa do `SKIP LOCKED`. O POST da seção anterior já respondeu ao comerciante antes deste loop.

3. Consolidação

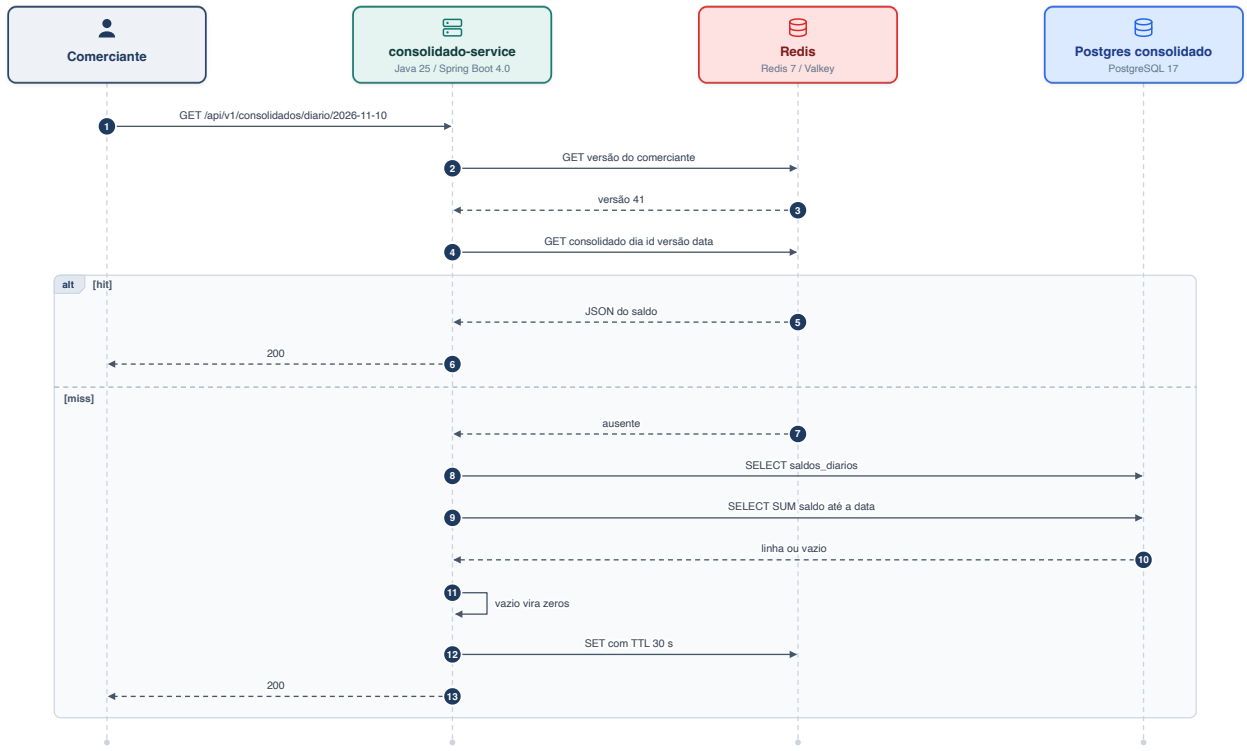
Consolidação



O `INCR` da versão acontece depois do commit do saldo. Se o Redis falhar nesse momento, a versão antiga continua válida até o TTL de 30 s. Se o commit do banco falhar, a mensagem não é apagada e volta após o visibility timeout.

4. Consulta com cache hit e cache miss

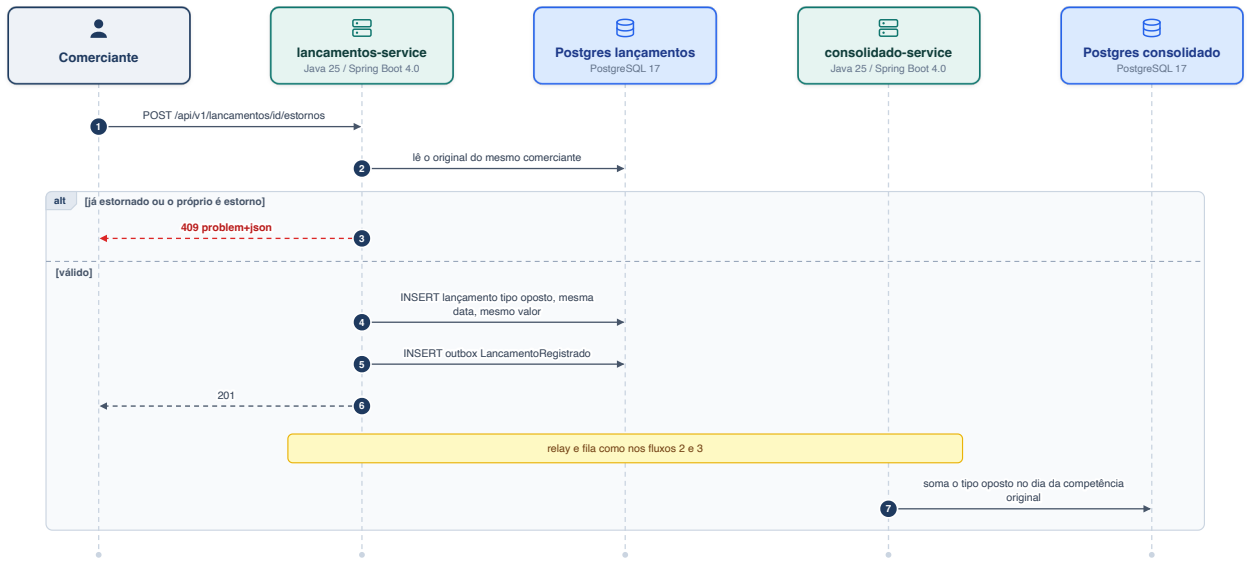
Consulta com cache hit e cache miss



O dia sem movimento não é miss eterno contra a tabela vazia: o serviço cacheia também a resposta zerada, para um cliente que insiste no mesmo dia não martelar o `SUM`.

5. Estorno

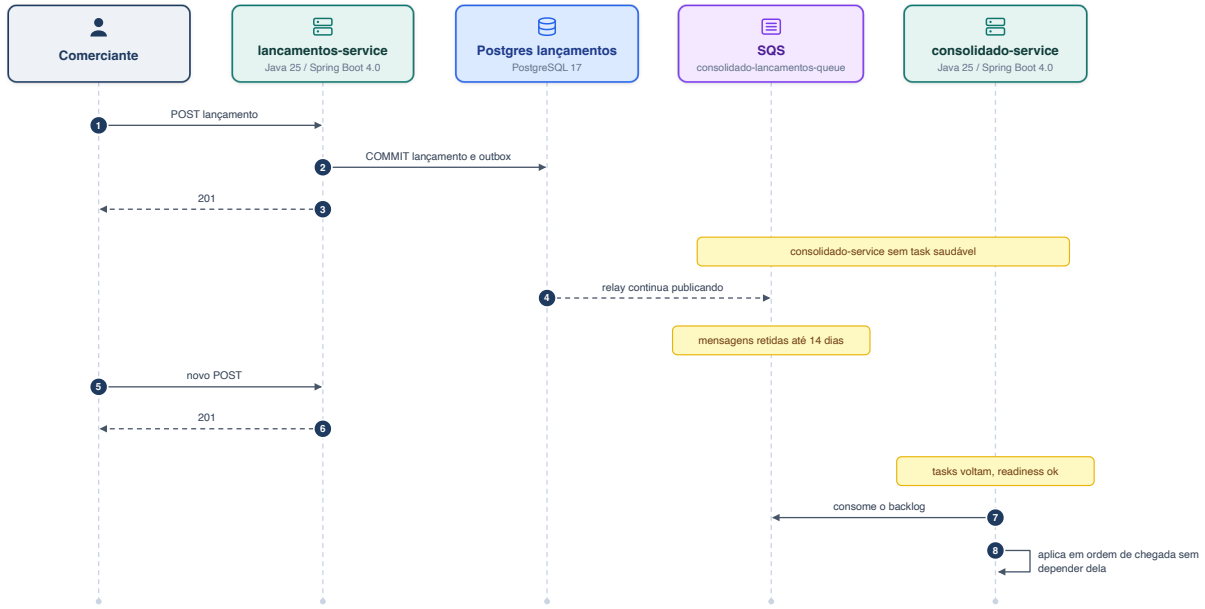
Estorno



O saldo do dia corrente só muda se a competência do original for o dia corrente. Estorno de um crédito de ontem corrige ontem.

6. Consolidado fora e a recuperação

Consolidado fora e a recuperação



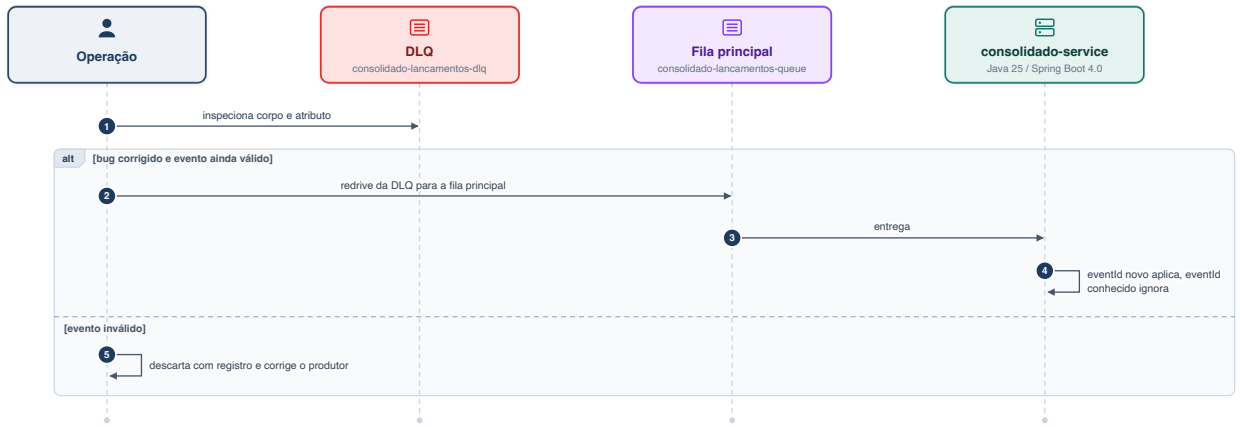
Durante a janela, `GET` no consolidado falha (entra no error budget de 99,5%). `POST` e `GET` de lançamentos seguem (SLO de 99,9%). Não há retry do cliente contra o consolidado embutido no POST.

Drenagem da premissa de 20 evt/s durante 1 h: 72.000 mensagens. Duas tasks a ~100 aplicações/s levam cerca de 6 minutos para alcançar o fim da fila. O p95 de 5 s vale para o regime estável, não para o catch-up depois de uma hora parado — o runbook trata esse catch-up como degradação esperada e medida pela idade da mensagem.

7. Reprocessar a DLQ

Uma mensagem chega na DLQ depois de 5 recebimentos. Causas típicas: JSON incompatível, `eventVersion` desconhecida, bug no `UPSERT`, poison message.

Reprocessar a DLQ



Redrive é seguro por causa do `eventId`. Não se edita a mensagem para "fazer passar". Se o contrato estiver errado, a correção é `eventVersion` nova e consumidor que a entende.

8. O que cada fluxo garante

Fluxo	Garantia	O que não garante
POST	Durabilidade do lançamento no commit	Saldo já visível
Relay	Publicação eventual com retentativa	Latência fixa se o SNS estiver degradado
Consumo	Soma única por `eventId`	Ordem entre eventos
GET com cache	Leitura rápida da última geração cacheada	Leitura monotonicamente fresca em todo miss concorrente
Queda do consolidado	POST segue	GET do saldo segue
DLQ	A fila principal não trava num poison message	O saldo daquele evento, até o redrive

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/PADROES-DE-INTEGRACAO.MD

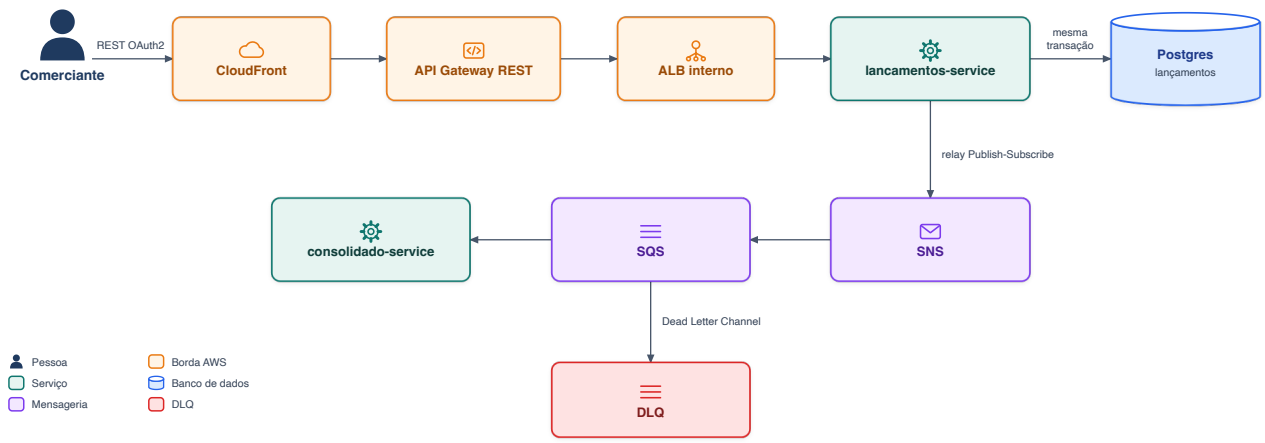
Padrões de integração

Catálogo aplicado, não um glossário de Enterprise Integration Patterns. Cada padrão abaixo existe num trecho concreto do caminho comerciante → borda → serviço, ou do caminho lançamentos → consolidado. O contrato (JSON, paths, atributos) continua no blueprint. O porquê da fila está no ADR 0005; o porquê da outbox, no ADR 0004.

Dois canais, e só dois:

1. **Síncrono**, comerciante para cada serviço, REST com OAuth2.
2. **Assíncrono**, lançamentos para consolidado, SNS e SQS. Não há HTTP entre os dois serviços. Essa ausência é o RNF crítico, não uma omissão.

Padrões de integração — os dois canais



1. Request-Reply síncrono com OAuth2

O comerciante chama `POST /api/v1/lancamentos` ou `GET /api/v1/consolidados/...` e espera a resposta na mesma conexão. Na AWS o caminho é CloudFront → API Gateway REST regional → VPC Link v2 → ALB → serviço (ADR 0031). O token é Bearer JWT, emitido pelo Cognito (ou pelo Keycloak no laboratório). O authorizer do gateway recusa token inválido ou escopo errado antes da VPC. O serviço valida o mesmo JWT de novo, com o JWKS do issuer, e lê `comerciante_id` do claim. O laboratório não tem gateway: o binário continua sendo o resource server (ADR 0011).

O gateway é o API Gateway pattern desta borda: um contrato `/api/v1`, throttle por método (o GET do consolidado acima de 50 req/s em hml e prod) e, para consumidor de integração, usage plan com API key. Cliente final não leva API key. `api_key_required` nasce falso em toda rota e só liga na rota que o tfvars marcar. A cota do usage plan é o teto daquele consumidor, distinto do rate-based por IP do WAF.

O reply de erro do serviço é RFC 9457 (`application/problem+json`). O 401 do authorizer e o 400 do request validator são a resposta do gateway, antes do serviço. O reply de sucesso devolve o recurso e o header `X-Correlation-Id`. Cliente que não recebe resposta por timeout trata o POST como incerto e repete com a

mesma `Idempotency-Key`. Sem essa chave, repetir o POST cria outro lançamento: o canal síncrono não deduplica sozinho.

Não há Request-Reply entre os serviços. Um GET do consolidado de volta ao lançamentos, no meio da consolidação, recolocaria o relatório no caminho crítico da escrita.

2. Publish-Subscribe Channel

O relay publica em `LANCAMENTOS_TOPIC_ARN`. O tópico é o canal. A assinatura que existe hoje entrega numa fila. Uma segunda assinatura (outro contexto, um data lake) ganha outra fila e outra role, sem o publicador saber o nome do consumidor. É o motivo de SNS na frente de SQS, e não uma fila em que o lançamentos escreve direto: o publicador conhece o tópico, não os interessados.

O corpo, com raw message delivery, é o JSON da linguagem publicada. O tópico não transforma payload.

3. Point-to-Point e Competing Consumers

A fila `CONSOLIDADO_QUEUE_URL` é point-to-point: uma mensagem é processada por um consumidor lógico (o serviço de consolidado). Várias tasks fazem long polling ao mesmo tempo. O SQS entrega cada mensagem a uma task por vez (visibility timeout de 60 s). Isso é Competing Consumers: a escala de drenagem é o número de tasks, não um consumidor único com thread pool interno.

O bulkhead do Resilience4j (ADR 0023) limita quantas mensagens uma task processa em paralelo, para o pool JDBC não virar o concorrente real. Sem esse teto, Competing Consumers “funcionando bem” esgota `max_connections` no pico de catch-up.

4. Transactional Outbox

O caso de uso grava o lançamento, a linha de idempotência e a linha de `outbox_eventos` na mesma transação. O 201 sai quando esse commit volta. O relay, noutro agendamento, lê com `SELECT … FOR UPDATE SKIP LOCKED` e publica. Dual-write (commit e `Publish` em sequência sem outbox) perde o evento quando o processo morre no meio, ou segura o 201 até o SNS responder. Os dois resultados violam o desenho.

O relay não é um terceiro serviço. É um agendador dentro do processo de lançamentos. Duas tasks não levam a mesma linha por causa do `SKIP LOCKED`.

Colunas de contexto, gravadas no POST e relidas pelo relay: `trace_parent`, `trace_state`, `correlation_id`. O detalhe de propagação está no documento 07.

5. Idempotent Receiver

Entrega at-least-once produz duplicata. O consumidor insere `eventId` em `eventos_processados` na mesma transação do `UPSERT` do saldo. Chave duplicada: a transação termina sem somar de novo e a mensagem é apagada. Reprocessar a DLQ é seguro por causa disso.

O POST tem o mesmo padrão no canal síncrono, com outra chave: `Idempotency-Key` mais a impressão digital do corpo. Chave igual e corpo diferente responde 409. São dois receivers idempotentes, com chaves que não se misturam. `eventId` não substitui `Idempotency-Key`, e o contrário também não.

6. Dead Letter Channel

Depois de `maxReceiveCount` 5, a mensagem vai para `consolidado-lancamentos-dlq`. A fila principal segue. Sem esse canal, uma mensagem inválida volta para sempre, queima receive e esconde o erro atrás de um backlog que não drena.

A DLQ não é reprocessada por código. O runbook está no documento 07: corrigir a causa, redrive, deixar o receiver idempotente absorver a repetição. Versão de evento desconhecida vai para a DLQ de propósito. Alargar o consumidor "para não falhar" engoliria uma quebra de contrato.

7. Message Expiration e retenção

A fila principal retém a mensagem por **14 dias**. Esse é o prazo em que o consolidado pode ficar fora sem perder evento. Não é um TTL de negócio do lançamento: o fato já está no Postgres de lançamentos. Passados 14 dias sem consumidor, a mensagem expira e o saldo daquele evento só volta por reconciliação, não pela fila. O alarme de idade existe para ninguém descobrir a expiração no décimo quinto dia.

Visibility de 60 s é o prazo de "esta tentativa ainda está com um consumidor". Processar leva milissegundos. Sessenta segundos cobrem um GC ruim sem devolver a mensagem cedo, o que duplicaria trabalho (o receiver aguenta a duplicata, mas o alarme de lag mente).

A outbox tem outra retenção, operacional: linha com `publicado_em` preenchido é apagada depois de 7 dias. Linha não publicada não expira por job. O documento de desempenho fixa essa limpeza.

8. Published Language e Anticorruption Layer

A linguagem publicada é o JSON `LancamentoRegistrado` versão 1: `eventId`, `eventType`, `eventVersion`, `occurredAt`, `lancamentoId`, `comercianteId`, `tipo`, `valor` (string decimal), `moeda`, `dataLancamento`, `lancamentoEstornadoId`. Sem descrição e sem categoria. Isso é minimização, não economia de bytes: a descrição pode carregar dado pessoal, e o consolidado não precisa dela para somar.

O consumidor não desserializa esse JSON direto num agregado. O `TradutorEventoLancamento` é a camada anticorrupção: traduz o JSON para o comando interno `AplicarLancamento`. Campo desconhecido é ignorado. `eventVersion` diferente de 1 falha a mensagem, que cai na DLQ depois das tentativas. Assim o modelo de saldo não importa tipo do produtor, e o produtor não importa tabela do consumidor.

9. Event-Carried State Transfer, não Event Notification

Há duas formas de contar o fato ao consolidado.

Event Notification. A mensagem leva só `eventId` (e talvez `lancamentoId`). O consumidor chama o lançamentos para buscar o valor. O relatório passa a depender da API de escrita estar no ar e do contrato HTTP. O RNF crítico deixa de valer no caminho inverso: o consolidado, ao voltar de uma queda, não drena a fila se o

lançamentos também estiver degradado. Além disso cria o acoplamento que a outbox existe para evitar.

Event-Carried State Transfer do agregado inteiro. A mensagem leva descrição, categoria, tudo. O consumidor vira uma segunda cópia do lançamento, com dado pessoal na fila, no disco do SQS e em qualquer assinante futuro. A fila deixa de ser um fato contábil mínimo e passa a ser um extrato.

O que foi escolhido. Transferência do **recorte que a projeção precisa para somar**, e nada mais. O consolidado aplica o saldo sem callback. Um estorno viaja como outro `LancamentoRegistrado`, com tipo oposto e `lancamentoEstornadoId` preenchido: o consumidor não tem regra especial de estorno. Se um dia o relatório precisar da descrição, a decisão se reabre com consciência de que isso copia dado pessoal — não se "acrescenta o campo porque já estava no banco".

10. Correlation Identifier

Três identificadores, com papéis que não se substituem:

Id	Quem cria	Para que serve
`correlationId`	A borda HTTP, a partir de `X-Correlation-Id` ou um UUID novo	Achar a jornada nos logs, inclusive depois da fila
`traceId` / `traceparent`	O tracer W3C no POST	Achar os spans no Tempo ou no X-Ray
`eventId`	O caso de uso, ao gravar a outbox	Idempotência de negócio. Não é id de trace

O `correlationId` vai no MDC, na coluna `correlation_id` da outbox e no atributo de mensagem `correlationId`. O consumidor reabre o MDC com ele. Um suporte que recebe o id do comerciante (o header volta na resposta) filtra log dos dois serviços sem precisar do trace.

11. Message Envelope e atributos

Raw message delivery está ligado: o corpo no SQS é o JSON da linguagem publicada, não o envelope SNS (`Message`, `MessageId`, `TopicArn` em volta). Um envelope em volta quebraria o consumidor e a ACL, que esperam o JSON da seção 6 do blueprint.

Metadado que não é fato de negócio vai em **message attribute**, tipo `String`:

Atributo	Conteúdo
`eventType`	`LancamentoRegistrado`
`traceparent`	W3C, gravado na outbox no POST
`tracestate`	Quando houver
`correlationId`	O mesmo UUID da requisição

Atributo permite filtro de assinatura no futuro (`eventType`) sem parsear o corpo, e tira o contexto de trace de dentro do contrato versionado. Colocar `traceparent` dentro do JSON obrigaria todo consumidor, inclusive um de BI, a conhecer telemetria. Atributo deixa o JSON estável.

O relay publica com esses atributos relendo a linha. O consumidor pede `MessageAttributeNames=All` e `MessageSystemAttributeNames=All` . Sem o `All` , o SQS devolve o corpo e omite o contexto, e o trace quebra no meio mesmo com a coluna preenchida.

12. Retry com backoff e jitter

Dois retries, em camadas diferentes.

Publicação. Resilience4j no cliente SNS do relay: Retry com backoff exponencial e jitter, TimeLimiter, e CircuitBreaker. O jitter existe para duas tasks não martelarem o SNS no mesmo milissegundo quando ele volta. Quando o circuito abre, o relay adia o lote: as linhas ficam na outbox com `proxima_tentativa_em` empurrado, até 60 s. Nada se perde, e o POST não participa.

Consumo. Falha transitória de banco no consumidor: Retry com backoff, dentro do visibility. Falha de domínio (versão desconhecida, JSON inválido) não retenta no aplicativo: devolve a falha, o SQS conta o receive, e a DLQ decide. Retentar 4xx de negócio só adia a DLQ.

O cliente do caixa, do lado de fora, não retenta 4xx. 409 de idempotência não melhora na segunda tentativa com outro corpo. 401 não melhora sem token novo.

13. Circuit Breaker

Não há circuito entre lançamentos e consolidado, porque não há chamada entre eles. Os circuitos que existem:

Onde	O que abre	O que acontece aberto
Relay, cliente SNS	Falha ou lentidão do publish	O lote fica na outbox. O 201 já foi embora.
Consolidado, cliente Redis	Falha ou lentidão do cache	A leitura cai no Postgres. O saldo continua correto, mais lento.
Deploy ECS	Task nova sem readiness	Rollback. É outro mecanismo, com o mesmo nome. ADR 0020.

O estado do circuito entra em `/actuator/health` sem derrubar readiness. Redis aberto não pode tirar a task do ALB: isso transformaria uma degradação de latência em indisponibilidade.

14. Garantia de entrega e ordenação

A garantia é **at-least-once**. O SNS pode entregar duas vezes. O relay pode publicar e morrer antes de gravar `publicado_em` , e publicar de novo. O SQS Standard pode entregar fora da ordem em que o SNS recebeu.

Por que não FIFO. A aplicação do saldo é uma soma. Soma é comutativa: crédito de 10 e débito de 4 no mesmo dia produzem o mesmo saldo em qualquer ordem. A duplicata é resolvida por `eventId`, não por ordenação. FIFO cobraria mais, imporia limite de throughput por message group e exigiria escolher a chave do grupo (`comercianteId` serializaria o caixa de um comerciante atrás de uma mensagem lenta). Nada disso melhora o saldo. FIFO voltaria a ser argumento se um evento futuro não fosse comutativo (saldo que depende do evento anterior, máquina de estados). Esse evento não existe. O ADR 0005 registra a mesma escolha pelo preço; aqui o motivo é a álgebra do domínio.

O que FIFO não consertaria: a janela entre o commit do lançamento e o publish. Essa janela é a consistência eventual, com meta p95 de 5 s, e é aceita.

15. Versionamento de contrato

API HTTP: prefixo `/api/v1`. Campo novo na resposta é aditivo; cliente ignora o que não conhece. Quebra (tirar campo, mudar tipo de `valor`) é `/api/v2` com os dois prefixos no ar até o último cliente antigo sair. O ALB roteia os dois prefixos para o mesmo target group enquanto a convivência durar.

Evento: `eventVersion`. Campo novo entra na versão 1 se for aditivo e o consumidor antigo puder ignorá-lo. Mudança que o consumidor precisaria entender (outro significado de `tipo`, valor em centavos inteiros) é versão 2, publicada em paralelo durante a convivência, com a ACL aceitando as duas e a versão desconhecida indo para a DLQ. Não se reescreve o histórico: evento já publicado permanece na versão em que nasceu.

16. Contract testing

O produtor e o consumidor moram em repositórios Maven separados. Um teste que só sobe os dois no Compose pega divergência tarde. O contrato a travar é o JSON da linguagem publicada mais os atributos obrigatórios (`eventType`, e a presença de `correlationId` / `traceparent` quando o publish sai do relay).

Pact (consumer-driven) cabe: o consolidado publica a expectativa do JSON v1 e dos atributos; o lançamentos verifica essa expectativa no CI, sem subir o outro processo. Até o Pact entrar no pipeline (documento 14), o mínimo que já segura o contrato é o teste de integração do produtor, que serializa o evento, e o teste da ACL, que recusa versão desconhecida. Os dois usam o mesmo fixture. Fixture copiado à mão nos dois projetos é o risco: uma mudança num lado só passa no teste local e quebra na fila. O fixture deve ter uma origem só, ou o Pact substitui a cópia.

17. Critérios de segurança para consumir

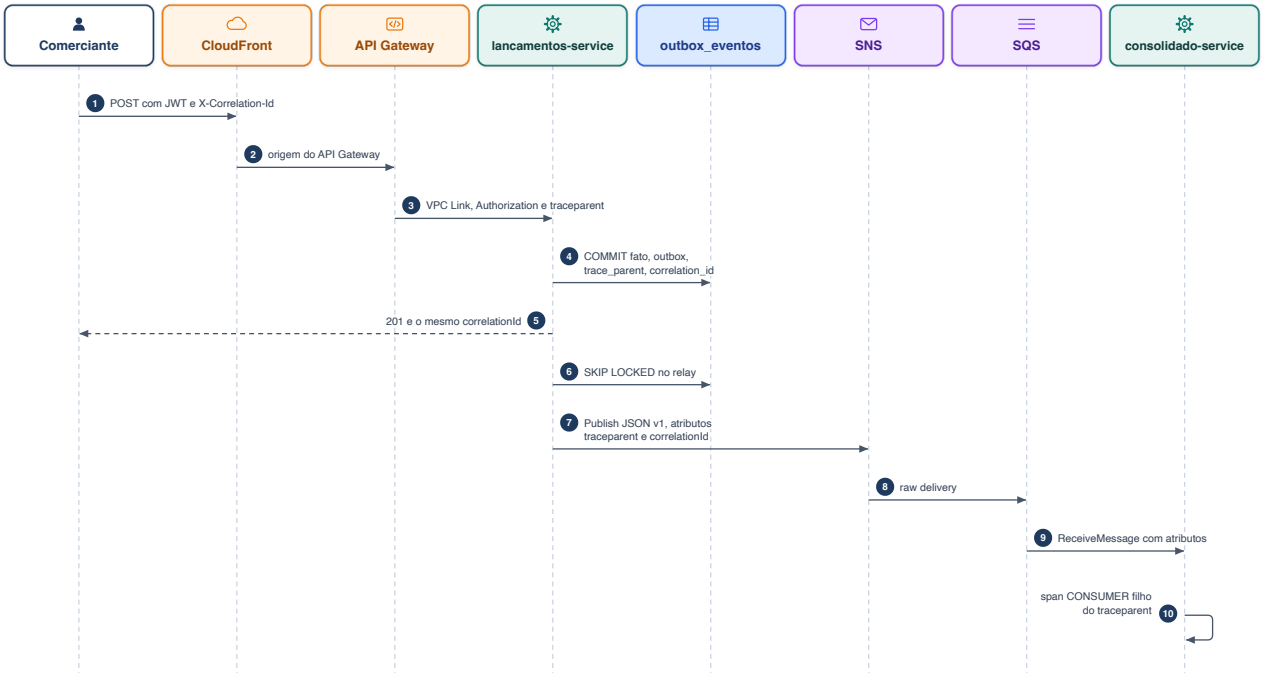
O detalhe de STRIDE e OWASP está no documento 06. O mínimo para um integrador novo:

Controle	HTTP	Mensagem
Identidade	Bearer JWT do issuer combinado. API key só no consumidor de integração cadastrado no usage plan; o cliente final não leva chave.	IAM. Não existe endpoint “injeta evento”.

Controle	HTTP	Mensagem
Autorização	Escopo no authorizer do gateway e de novo no serviço. Leitura não recebe write.	`sns:Publish` só na role de lançamentos, no ARN do tópico. `sqs:ReceiveMessage` e `DeleteMessage` só na role de consolidado, no ARN da fila. Segundo consumidor ganha fila e role novas.
Política de recurso	O ALB não é a autorização.	Policy do tópico e da fila recusa principal que não seja a role nomeada. `Resource` é o ARN, não `*`.
Tenant	`comercianteId` no corpo é ignorado. O claim do token manda.	Só o produtor preenche `comercianteId`. O consumidor não "corrige" tenant.
KMS	Segredo de banco na CMK do serviço.	SSE-KMS na fila, no tópico e na DLQ. A role de consumo tem `kms:Decrypt` nessa chave, não na chave do outro serviço.
TLS	TLS 1.2+ no CloudFront. JDBC com `sslmode=require`. Redis com TLS na AWS.	API da AWS em HTTPS, pelo VPC endpoint em hml e prod.
Validação	Bean validation na borda HTTP e invariante de novo no domínio.	ACL valida versão, tipo fechado, valor decimal positivo, UUID. JSON inválido não é aplicado pela metade.
Segredo no payload	Token não vai para log nem query string.	Senha, descrição e token não entram no evento nem no atributo.

Assinatura HMAC do corpo não está ligada. A policy da fila, o raw delivery dentro da conta e a rede fechada são o controle. HMAC passa a ser necessário se um segundo produtor puder publicar no mesmo tópico ou se a fila deixar de ser privada. Até lá, seria mais uma chave para girar, sem um agressor no caminho que a chave impedisse.

Correlação ponta a ponta — do POST ao consumidor



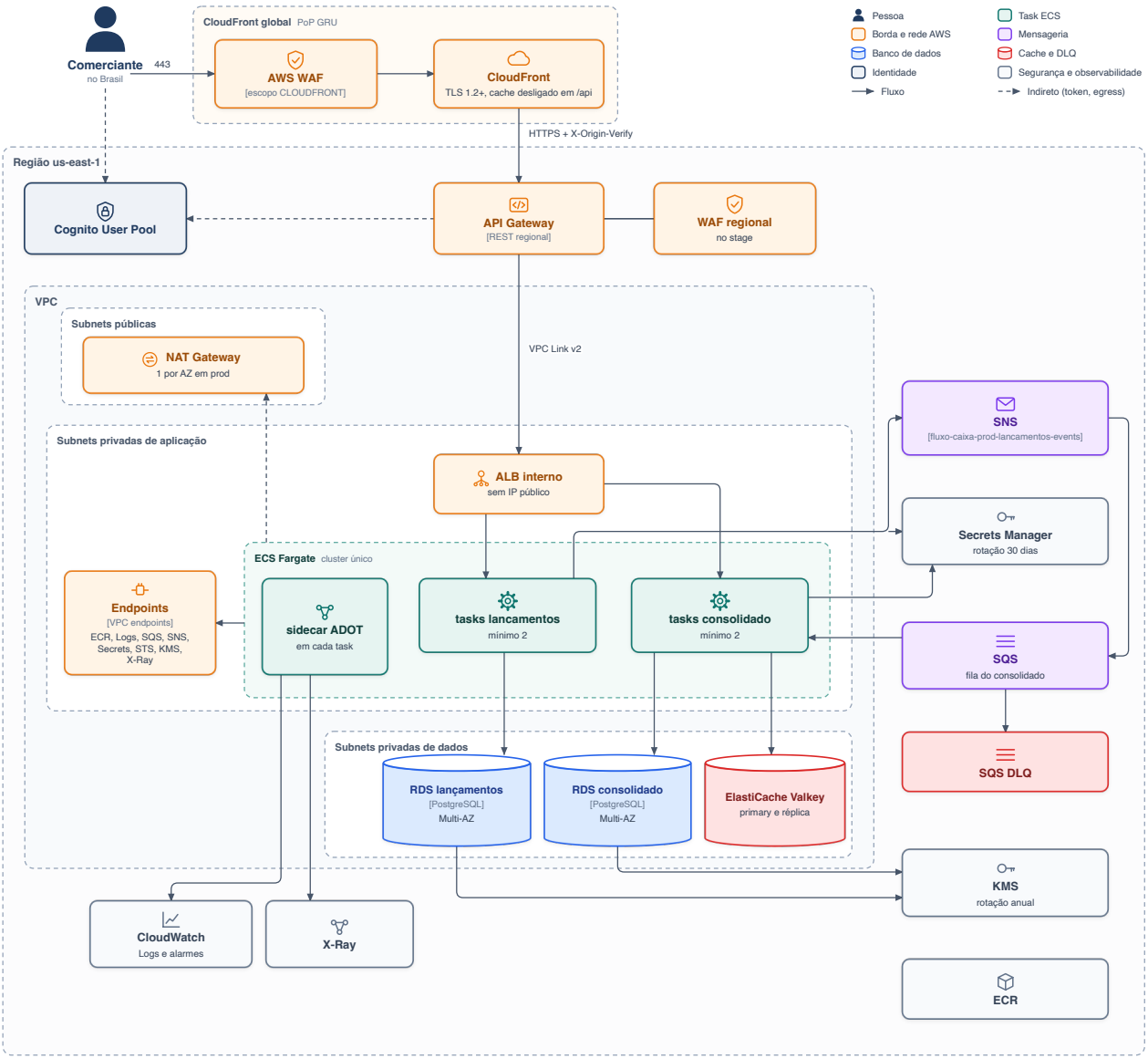
FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/DEPLOYMENT-AWS.MD

Deployment na AWS

Região `us-east-1` (ADR 0021). Três ambientes, `dev`, `hml` e `prod`, com o dimensionamento da seção 15 do blueprint. A topologia abaixo é a de **produção** com API Gateway ligado (ADR 0031): VPC em três zonas, NAT por zona, endpoints de interface, CloudFront com WAF, REST API regional, VPC Link v2, ALB interno, ECS Fargate, dois RDS Multi-AZ, ElastiCache com réplica, SNS/SQS/DLQ, Cognito, Secrets Manager com rotação e ADOT. Hml usa duas zonas, um NAT e endpoints. Dev usa um NAT, sem endpoint, e Fargate Spot. O dia a dia do desenvolvedor é o Docker local, com custo de nuvem zero.

1. Diagrama

Deployment na AWS — produção



S3 entra como gateway endpoint (sem custo horário) para o pull de camadas via ECR. A lista de interface é: ECR api, ECR dkr, CloudWatch Logs, SQS, SNS, Secrets Manager, STS, KMS e X-Ray. KMS e X-Ray não estão na lista mínima da seção 12 do blueprint; existem para a CMK e o sidecar ADOT não saírem por NAT. O agente de integração confere o Terraform.

CloudFront é global. O desenho o coloca ao lado da região porque o Web ACL de escopo `CLOUDFRONT` e o certificado ACM moram em `us-east-1`, e o PoP que o comerciante usa é GRU.

2. Rede

Camada	Quem mora	Regra
Borda	CloudFront, WAF CLOUDFRONT, API Gateway, WAF regional	Única URL pública é o CloudFront. `/api/*` sem cache. O `execute-api` existe e o WAF regional o fecha.
Pública da VPC	NAT	Sem rota de entrada. O ALB não está aqui.
Aplicação	ALB interno, VPC Link, tasks, endpoints	O security group do ALB aceita o prefix list do CloudFront e o security group do VPC Link. Task sem IP público. Egress para RDS, Redis, endpoints e NAT.
Dados	RDS, ElastiCache	5432 só do security group do serviço dono do banco. 6379 só do security group do consolidado.

Três AZs em prod permitem perder uma zona e ainda agendar task nas outras duas. O mínimo de duas tasks usa `spread` por AZ. NAT por AZ evita que a queda de uma zona leve o egress das outras. O preço está no custo: três NAT em `us-east-1` são USD 99/mês; os 27 endpoints são USD 197/mês.

Dev não cria endpoint: o tráfego AWS sai pelo único NAT. Hml cria os endpoints em duas zonas, para o ensaio da rede privada não acontecer pela primeira vez em prod.

3. Borda

ADR 0022 e ADR 0031.

- `PriceClass_All`, HTTP/2 e HTTP/3, TLS `TLSv1.2_2021`, ACM em `us-east-1`.
- WAF de escopo CLOUDFRONT: Common, Known Bad Inputs, SQLi, IP Reputation, rate-based por IP. AnonymousIpList em count no dev. Log do WAF com corpo e `Authorization` redigidos.
- Com `enable_api_gateway = true`, `/api/v1/lancamentos*` e `/api/v1/consolidados*` usam a origem HTTPS do API Gateway (`origin_path` `/v1`, header `X-Origin-Verify`, origin request policy `AllViewerExceptHostHeader`). O comportamento default continua 404 na função, sem chamar origem.
- REST API regional, stage `v1`, authorizer Cognito por escopo, VPC Link v2 no ALB. WAF regional no stage exige o header. Rotação do valor em 30 dias (90 em dev), mantendo o anterior.
- ALB interno, listener HTTP 80. O TLS da internet termina no CloudFront; o trecho VPC Link → ALB é HTTP na rede da AWS. Regras `/api/v1/lancamentos*` e `/api/v1/consolidados*`, mais o prefixo `/v1` reescrito quando o gateway está ligado. Health check: `GET /actuator/health/readiness`, direto no tar-

get, sem passar pelo listener. O access log vai para S3 com SSE-S3: o ELB não aceita CMK nesse bucket.

- O limite rate-based fica **acima** do pico legítimo de um IP. 50 req/s é o pico do serviço, não de um cliente. A ordem de partida é 30.000 requests por IP a cada 5 minutos. O throttle do gateway no GET do consolidado fica acima desses 50 em hml e prod.
- Shield Standard acompanha o CloudFront. Shield Advanced fica de fora.
- `/actuator` e o Swagger não são rota do gateway nem do ALB.
- `enable_api_gateway = false` tira o gateway e devolve a VPC Origin do ADR 0022.

4. Computação

Serviço ECS	Task em prod	Mínimo	Máximo	Capacidade
`lancamentos`	0,5 vCPU, 1 GiB, ARM64, sidecar ADOT incluso, on-demand	2	10	CPU 60% e request por target
`consolidado`	1 vCPU, 2 GiB, ARM64, sidecar incluso, on-demand	2	10	CPU 60%, request por target e backlog da fila

O blueprint (seção 15) fixa o máximo de prod em 10 para os dois. Dev: 0,5 vCPU / 1 GiB, mínimo 1, máximo 2, Fargate Spot, nos dois serviços. Hml: lançamentos 0,5/1 e consolidado 1/2, mínimo 1, máximo 3, on-demand.

Deployment circuit breaker com rollback (ADR 0020).

A task de 1 vCPU do consolidado é maior do que o pico de 50 req/s exige (ρ de cerca de 17% com duas tasks). O tamanho existe para caber o sidecar e o JIT. O lançamentos fica em 0,5 vCPU porque o caminho de escrita é mais curto.

A raiz do container é somente leitura. O `/tmp` é um volume do Fargate e nasce como root. O entrypoint da imagem faz `chmod 1777` e desce para o usuário `app`; sem isso o Tomcat não cria o diretório temporário e a task morre no start.

5. Dados

Recurso	Produção	Papel na falha
RDS `lancamentos`	PostgreSQL 17, `db.t4g.medium`, Multi-AZ, gp3 50 GB, KMS, PITR, senha gerenciada	Standby noutra AZ. Falha do consolidado não o alcança.
RDS `consolidado`	Igual, instância separada. Réplica de leitura parametrizável, default 0	Restaura sem restaurar a trilha de lançamentos.
RDS Proxy	Parametrizável, default desligado	Não substitui o wrapper de segredo. Documento 12.

Recurso	Produção	Papel na falha
ElastiCache	Valkey, `cache.t4g.small`, uma réplica, failover, TLS, sem AUTH token	Perda do primário promove a réplica. Perda do cache degrada para o RDS.
Secrets Manager	Um segredo por RDS, rotação de 30 dias em prod e hml, 90 em dev	O wrapper rebusca. A senha não está na imagem nem no Git.

RPO ≤ 5 min vem do PITR. RTO ≤ 1 h é o tempo de restaurar dentro da região, ensaiado em game day. Não é failover de região.

`DB_READER_URL`, quando a réplica existir, manda só transação `readOnly` para o reader. O consumidor da projeção permanece no primário (ADR 0030).

6. Mensageria e identidade

- SNS, SQS e DLQ com SSE-KMS. Policy: a role de lançamentos publica; a role de consolidado consome. A policy do tópico não inclui `sns:Unsubscribe`: o SNS rejeita essa action. Prefixo `fluxo-caixa-prod-` (e `dev` / `hml` nos outros ambientes). O serviço lê ARN e URL, não o nome.
- Cognito User Pool no plano Essentials, resource server `fluxo-caixa`, escopos `lancamentos:write`, `lancamentos:read`, `consolidado:read`. Prefixo de autoridade `fluxo-caixa/`.
- Pre Token Generation Lambda (v2, plano Essentials) insere `comerciante_id` e os três escopos do resource server no access token. O `InitiateAuth` sozinho não traz escopo customizado; sem o acréscimo o API Gateway responde 403. Sem o claim o serviço responde 401. O domínio hospedado é `fluxo-caixa-<env>-<account_id>`.
- JWKS buscado pelo serviço. Não há PEM na imagem.

7. Observabilidade na implantação

Sidecar ADOT: traces para X-Ray, métricas para CloudWatch. Log JSON com `traceId`, `spanId` e `correlationId`. O relay republica o `traceparent` guardado na outbox. Detalhe e as queries no documento 07.

Alarmes (5xx, p99, profundidade e idade da fila, DLQ, outbox, circuito aberto, `RotationFailed`) publicam num tópico SNS de operação. Container Insights desligado em dev, ligado em hml, enhanced em prod. Retenção de log: 7, 14 e 30 dias.

8. Escalabilidade

Escala horizontal até 10 tasks por serviço em prod. O consolidado também escala pelo backlog da fila. O próximo gargalo, se o pico deixar de ser 50 req/s, é o `SUM` do acumulado e o primário do Valkey, não o CloudFront. A resposta está no documento 12: janela SQL, depois réplica, bem depois outro read store. Nada disso é necessário para o requisito atual.

9. Disaster recovery

Cenário	Comportamento	RTO esperado
Task morta	ECS substitui. ALB tira do target group no readiness.	minutos
AZ morta	Task na outra AZ segue. RDS Multi-AZ promove. ElastiCache promove a réplica. NAT das outras AZs segue.	minutos para dado gerenciado
Mensagem poison	DLQ, serviço continua	operação humana para aquele evento
Corrupção lógica	PITR, no máximo 5 min atrás	até 1 h
Rotação de senha falha	Alarme `RotationFailed`. Wrapper rebusca. Runbook do ADR 0024.	minutos se o runbook rodar
Região `us-east-1` indisponível	Sem active-passive. O RTO de região não é o RTO de 1 h.	evolução, com decisão jurídica se o secundário sair dos Estados Unidos

Restauração é para uma instância nova. Não se restaura por cima da instância que ainda recebe escrita.

10. Ambientes

	dev	hml	prod
AZs	2	2	3
NAT	1	1	3
Endpoint de interface	não	2 AZs	3 AZs
Fargate	Spot	on-demand	on-demand
RDS	micro, Single-AZ, 20 GB	small, Single-AZ, 20 GB	medium, Multi-AZ, 50 GB
Backup	1 dia, sem deletion protection	7 dias	14 dias, PITR, deletion protection

State Terraform em S3, um prefixo por ambiente, lock por lockfile, em `us-east-1` (ADR 0029).

11. O que o diagrama não inclui de propósito

Conta separada por ambiente, Transit Gateway, Shield Advanced, multi-região e EKS. São camadas de uma organização com vários produtos ou com o gatilho do documento 13. Para dois serviços, aumentam o piso sem mudar o RNF. A comparação com `sa-east-1` está no documento de custos: a mesma produção sairia 1,92 vez mais cara, e a região vigente é a Virgínia com a salvaguarda do art. 33, não por desconhecimento do preço de São Paulo.